PROGRAMMERINGS OLYMPIADEN

Final 2025

Lösningsförslag

RGB Hydra

Av: Harry Zhang

Problem

Given a hydra with R red heads, G green heads, and B blue heads.

Use up to k moves to maximize number of red heads.

Insight

If we have at least one green head or blue head from the beginning, then we kan repeatedly cut off in the followin way:

blue -> green -> blue -> green -> …

In each move, we increase the number of red heads by 1

Solution

The answer is just R+k if there exist a green or blue head from the beginning.

If G == B == 0, we would have to sacrifice 1 move and 1 red head to get at least 1 blue and 1 green head.

Answer is in this case max(R,(R-1)+(k-1)).

Excursion

Problem idea: Joshua Andersson

Implemented by: Nils Gustafsson

Problem

Given N times when participants arrive to the bus, schedule when two buses should leave as to minimize the total time participants spend waiting.

Insight 1

Everyone must get on the bus. This also means that the last person must get on the bus, so one of the buses must be after the last person. We might as well schedule it right after the last person arrives.

Subtask 1

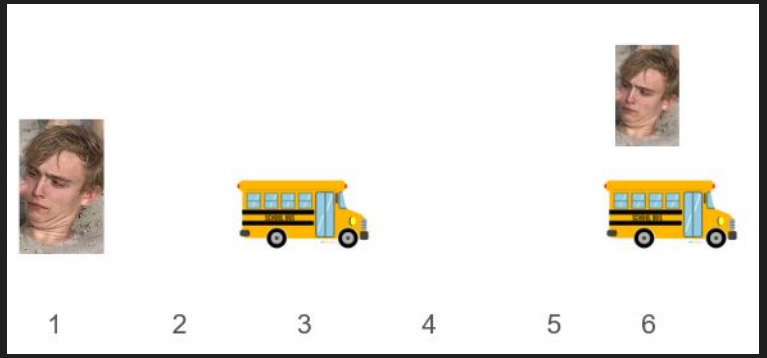
Because of insight 1, we only need to schedule one of the buses. In this subtask, there are only 300 points in time when it can leave. We can try each one and compute the time waited in O(N) and take the smallest of these. Make sure to sort the times first!

Subtask 2

In this subtask, there are up to 10^9 possible times for the second bus to depart. We need to reduce this. We can realize that it’s always optimal to let the second bus depart right as someone arrives, leaving us with O(N) candidates.
O(N^2) total.

Subtask 2

Instead of



We do

Subtask 3

We only need to test positions floor(n/2) and floor(n/2)-1

Can be found either via intuition, or using calculus

Can also use ternary search

Full solution

Placing the bus at t[i] requires us to compute the waiting time for people [0,i] and [i+1, n). For people [0,i], we can compute the value for each i all at once in O(N), a little similar to prefix sum. For [i+1,n), the waiting time for each person j will be t[N-1]-t[j]. Precompute this too, similar to suffix sum.

Now we can try placing the bus at each i in O(1)

Challenge

Solve the problem in O(N) (given that all t[i] are <= 10^9)

The Duckamyds

By: Theodor Beskow

Problem

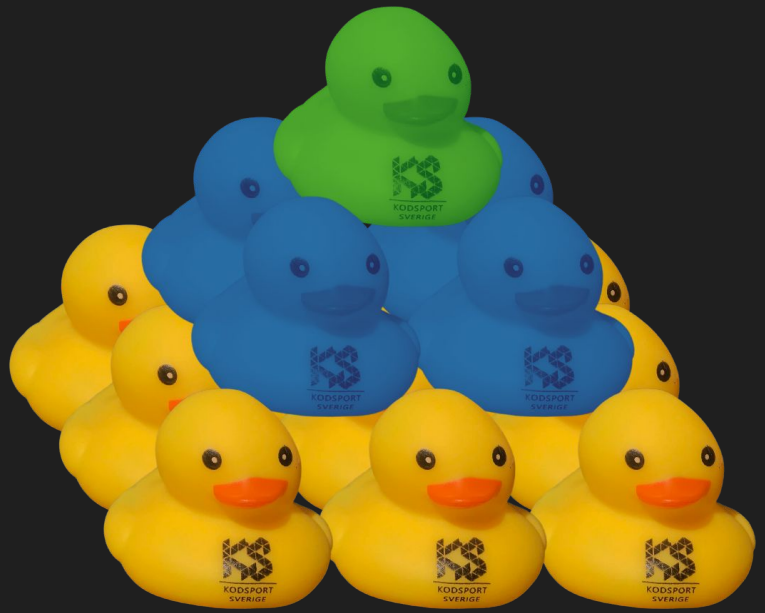
Given the amount of ducks of different colors and what color the different layers have to be.
How tall can the pyramid be?

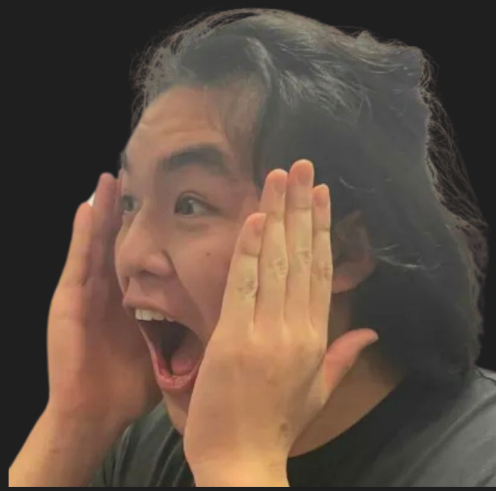
Your reaction when you found out Kodsport has duck merch |

v

$(H-i+1)^2$

Pyramid like this ->





Subtask 1: N = 1

only one type of duck

Keep adding another layer until you don't have enough ducks or M reached.

O(M)

Subtask 2: c[i] = i

No color used in more than 1 layer

For each layer compute how big a pyramid using that layer can be.
sqrt(ankor)+layer (0 indexerat)

Now find the largest k such that all the lowest k layers can build a pyramid k layers tall.

Subtask 3 and 4: sum(a[k]) <= 10^5 and N,M <= 2000

In subtask 3 the answer can max be ≈ 50, $(10^5)^{1/3}$
In subtask 4 the answer can max be = 2000, M

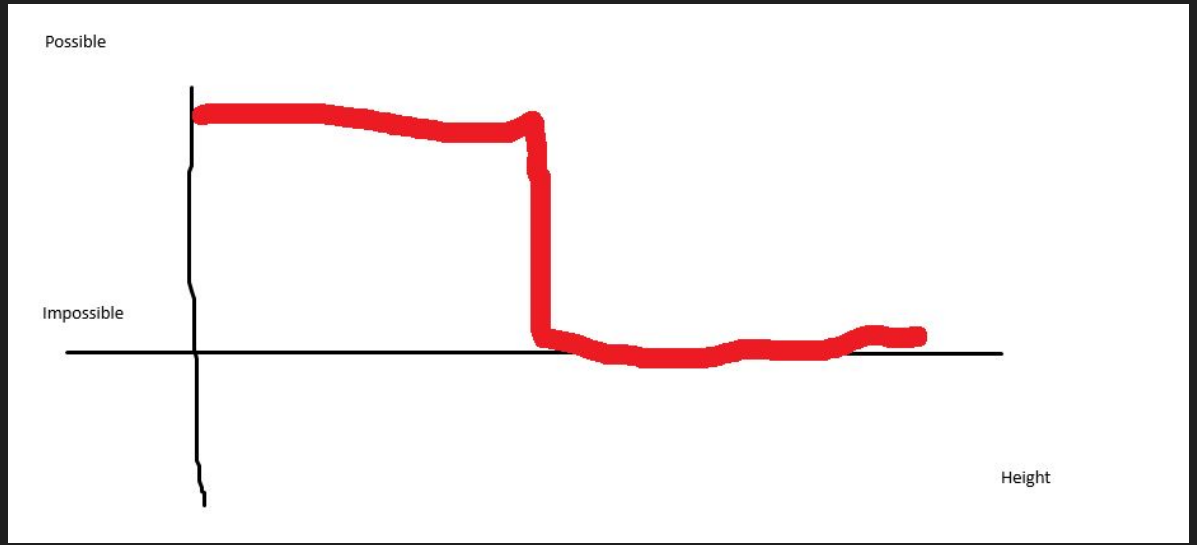
Try to build a pyramid of all possible heights until it's impossible

O(max_answer^2)

Full lösning

Insight:

It will be possible up to a certain value (the answer)



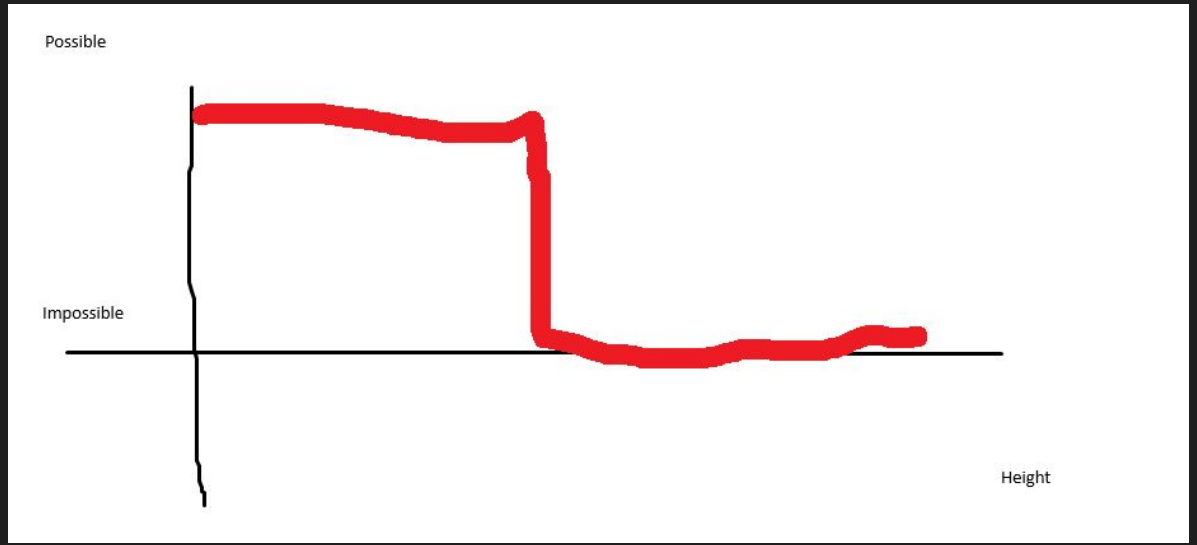
Full lösning

Solution:

Binary search the answer.

To check if possible is O(M)

O(M log(M))



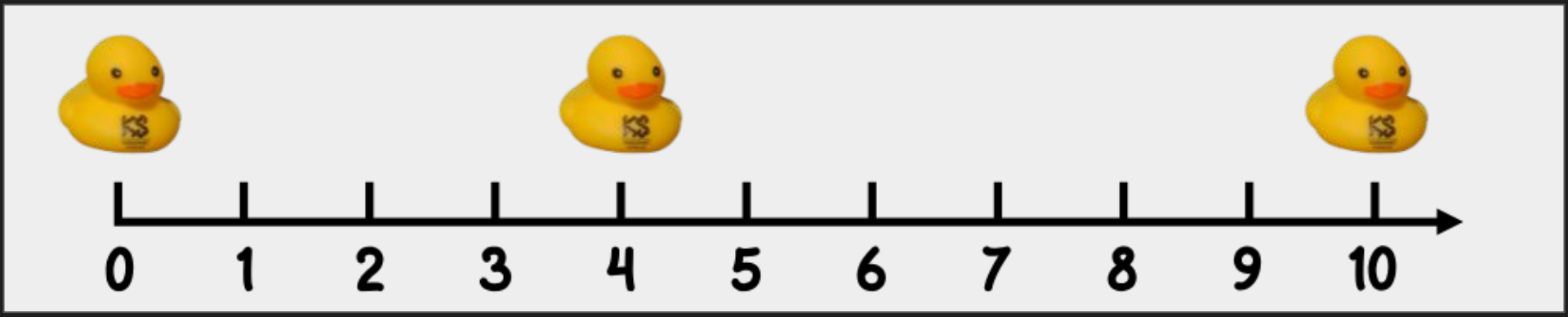
Quack

By: Harry Zhang

Problem:

There are N hidden ducks along a number line. You can ask questions in the form “What is the position of the closest duck to position x”.

Ask as few queries as possible to find all N ducks position.



Subtask 1

N = 2

Solution:
Ducks could only be on the interval [0,1e9].

Ask 2 queries from points away from where the ducks could be, for example x = 0, and x = 1e9.

Subtask 2

The ducks are right next to each other

Solution:
If we find the duck to the leftmost/rightmost, we know the positions of the rest of the ducks.

Same queries as subtask 1, but 1 query could also be enough.

Subtask 3/4

Q > 2N

How do we find the rest of the ducks?
Maybe find 1 duck at a time?

Solution:
We can for example use binary search.
First find the leftmost duck. Binary search to the right the smallest coordinate where the duck quacking back is not the same duck. Then we have found the duck closest to the leftmost duck. And so on…

Full solution

Q = 2N-1

We realise that if we have found 2 ducks, and want to check if there are any ducks between them, we can just query in the middle of them.

If there is a duck in the middle of them, then the duck that quacks back will not be any of the 2 ducks that we had from the beginning.

This can be implemented in O(N^2) time comp using inserting into lists, but to do it fast, you can do it recrusively.

Full solution

Q = 2N-1

Solution:

Find leftmost duck, call it a. Find right most duck call it b.

We can call solve(a,b), a function that tries to find ducks between a and b.

if solve(a,b) finds some duck a<x<b, then it recursively calls solve(a,x) and solve(x,b).

Number of queries used N (one per duck) + N-1 (between each pair of duck) = 2n-1.

Bartering

Problem idea: Joshua Andersson

Implemented by: Victor Vatn

Problem

There are N unique items and M trades. In each trade you take some items, possibly multiple of the same, and trade them for **exactly** one of another item. Your goal is to calculate how many of item 0 are needed to get item N-1.

Subtask 1

N = 3

Solution:
Repeatedly calculate the price of each item in terms of how many of item 0 is needed. Start by checking all trades that only have item 0 on the left side and result in item 1. We now know the cost of item 1.Then check all trades that result in item 2.

Subtask 2

$k_i = 1$

In this subtask there is only one unique item on the left side on each trade. This means we get a “normal” directed graph with weighted edges and we can do a djikstra.

(When we traverse an edge we multiply with the weight instead of adding it)

Subtask 3

The graph contains no “cycles”.

In this case we can do a topological sorting of the items and then process them in that order, the result of a later item won’t affect the earlier ones.

One way to implement this is to for each item keep track of all trades where it is on the left side, and keep track of all trades where it is on the right side. In the start you set the cost of all trades to 0, and the cost of item 0 to 1. Then when you process a node you first go through all trades where it is on the right side, to determine its cost. And then you go through all trades where it is on the left side, and add the cost times the coefficent to that trade

Subtask 4

The hard part of the problem is finding a good order of processing trades in. Using Bellman-Ford, we don’t need to think about order. The idea is to repeatedly iterate through all trades, and use them to improve prices until there are no more improvements. Of course, only use trades where we know the price of each part of the LHS in terms of paperclips.

Insight

We realise that there are no “negative” cycles. What this means is that there is no combination of trades that you can continue doing over and over again forever to get more items. This is because the result of each trade is always exactly one item. So it is never profitable to use a cycle, but we still have to deal with them.

Subtask 4 and Full solution

There being no negative cycles means we can solve this problem with a djikstra-like algorithm!

Let's consider the following solution to subtask 3:
Let's introduce the concept of "locking in" the price of some item u. This means that we have decided that the price of item u is some value, and cannot be improved. We lock in the price of every item exactly once (similar to djikstra).

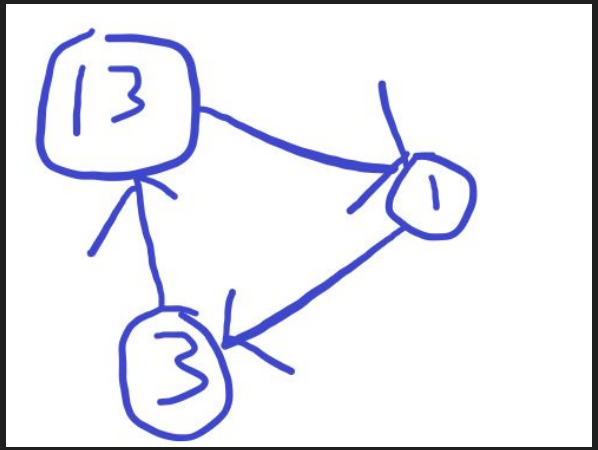
After we lock in the price of some item u, we check every trade that u is part of. If the act of locking in u leads to the price of all items in the LHS (left-hand side) of some trade being known, we calculate the price that the trade gives us for the resulting item.

Subtask 4 and Full solution

We don’t ever need to check this trade again, since we assumed that when we lock in items, their price cannot be improved. When some items has no more remaining trades, we can safely lock it in. To implement this efficiently, for each item we save which trades it is part of the LHS. For each trade, we maintain the number of unknowns in its LHS. When the number of unknowns becomes 0, we consider the trade. (There are also some details to handle finding out when some items has trades with it as the RHS, but those don’t generalize to the full solution)

Let’s extend this solution to solve the whole problem. The problem now is that there can be items that will never have 0 trades coming into them because of the cycles. However, recall that cycles are never useful.

Subtask 4 and Full solution



So we basically have the following situation. Which node should we choose to lock in? It can be proved that locking in the cheapest one (the one with cost 1) is always optimal.

So our algorithm becomes: maintain a heap over all nodes that could be locked in. Repeatedly pop the one with smallest value, let it be u. If u is already locked in, do nothing.
If u is not locked in, lock it in and check all trades u is part of. For every trade u is part of that now has 0 unknowns in its LHS, push the RHS and the price given by the trade to the heap.
Rinse and repeat until we know the price of item N-1.

Crack the password

Av: Joshua Andersson

Subtask 1

First, query ‘a’ * N. Now we know how many a’s there are. Start with a string aaa…aa with the correct number of ones. Then, try inserting b’s. Whenever they increase the LCS, we have found a correct b.

Subtask 2

First, query ‘a’ * N, ‘b’ * N, … Now we know how many there are of each character. Start with aaa..aaa (or b or c, take the one with most occurrences).

Now, for each position, try inserting each possible character. If we increase LCS, keep it, otherwise continue. Important: do not test the character you chose to start with (for example, if we start with aaa…aaa, we do not need to test a later)

~O(N^2K) queries

Subtask 3

Many possible solutions.

~65 points

Take the previous solution, but insert all of the same character at once. First take the character with least occurrences, second least etc.

If you don’t use best possible order of which characters to insert, you get around 50 points.

~O(NK) queries

With optimizations, can increase score further

100 points

Instead of inserting c's into abdefg, insert abdefg into cccccccc. Go backwards, and binary search how much of abdefg we can insert.

100 points

Try
ccccccccabdefg -> no
cccccccccefg -> yes
ccccccccdefg -> no

Thus, we now want to try to insert abd into cccccccccefg

100 points

Try
ccccccccabdcefg -> no
cccccccbdcefg -> yes

Now we want to try to insert a into cccccccbdcefg

100 points

Try
ccccccacbdcefg -> yes

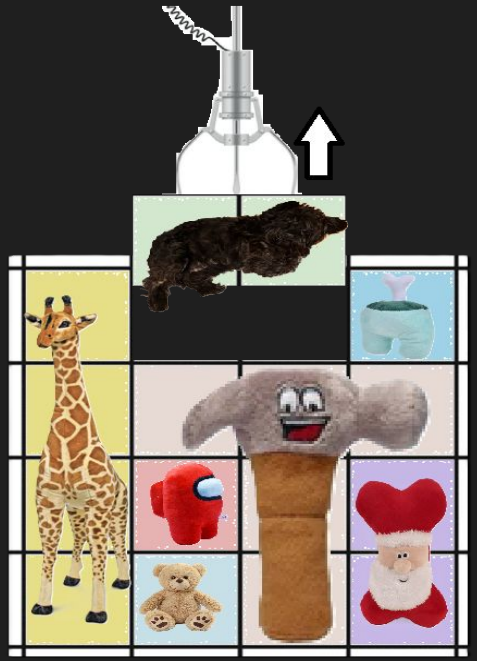
Now, move on to the next character.

In total, we do one binary search for each character. Upper bound Nlog_2(N) ~= 665 queries. In practice, using optimizations we get < 600 (although all solutions found so far can be made ~650 by finding test data that attacks that specific solution)

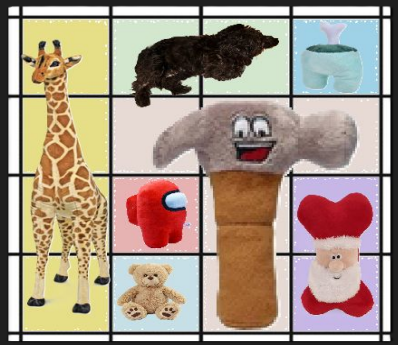
Gosedjur / Claw Machine

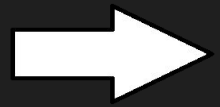
Av: Olle Lapidus

Problem



- Given an R x C grid describing a claw machine with N stuffed animals, answer for each animal how many others are “covering” it.
- Print N numbers, the i:th number corresponding to the i:th animal.
- If an animal is impossible to ever lift, print -1 as the answer for it.





1	3	3	6
1	4	4	4
1	7	4	5
1	2	4	5

“Interesting“ subtasks:

- Subtask 2: C = 1
- Subtask 5: N <= 2000
- Subtask 4: All animals have rectangular shape (ideas for this subtask are most similar to full solution!)

Subtask 2: C = 1

- All of the animals are stacked strictly on top of each other.
- The topmost one has answer 0, the next has answer 1, next 2, and so on.

Sample Input 2

```
4 5 1
2
1
1
3
4
```

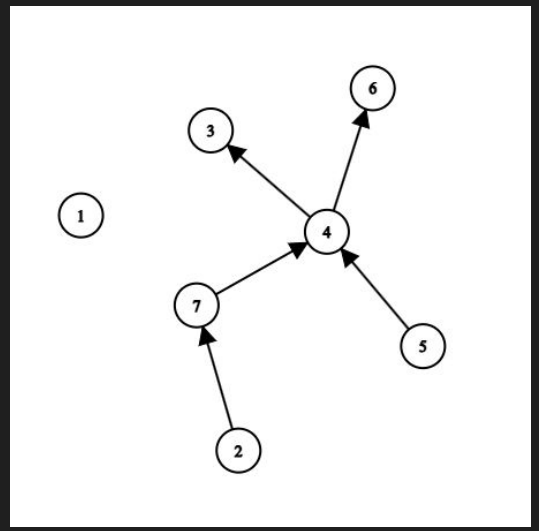
Sample Output 2

```
1 0 2 3
```

Subtask 5: N <= 2000

- The constraint is hinting that we want to find something $O(N^2)$.

1	3	3	6
1	4	4	4
1	7	4	5
1	2	4	5

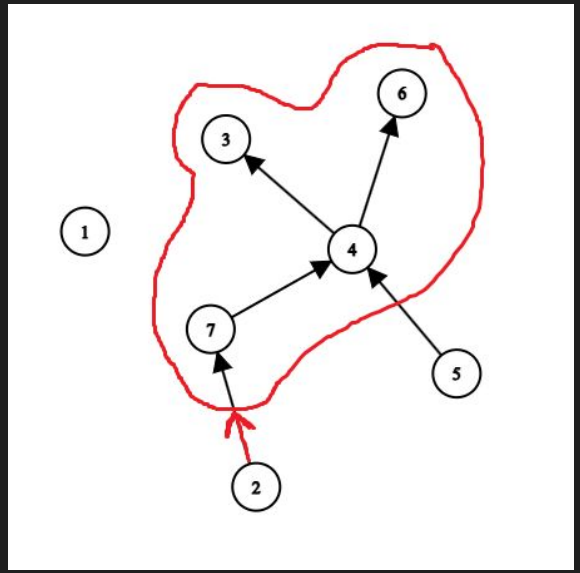


- Translate the grid into a graph, where there are directed edges whenever something is directly above (and touching).

Subtask 5: N <= 2000

- If the graph has a cycle somewhere, then all nodes in the cycle (and under/before the cycle) have answer -1.
- Otherwise, the answer for a certain node is the number of nodes that are reachable from it.
- Solution: For each node, do dfs
  - If end up in cycle when walking from here:
    - Answer = -1
  - Else:
    - Answer = number of nodes visited

- Node 2 can reach 4 nodes, so its answer is 4



Subtask 6: N <= 50000

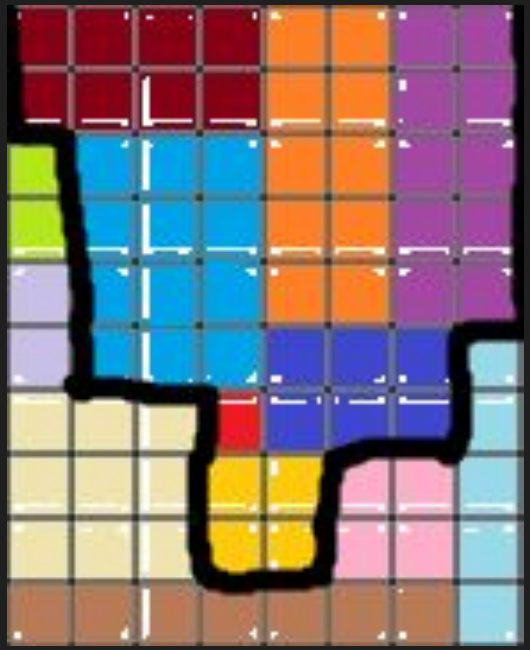
- Just do the same thing as N <= 2000 but use bitsets
- Assume node x has two neighbors: y and z. And assume the answer is not -1 for any of them.
- Then, set of reachable for x is union of set of reachable for y and set of reachable for z.
- bitset = representation of set. i:th bit is 1 if i:th element in the set, otherwise 0.
- bitset(x) = bitset(y) || bitset(z), use bitwise or to find union of two sets.

Subtask 4: All have rectangular shape

- Claim: Answer is never -1 in this case
- Proof: Obvious

Subtask 4: All have rectangular shape

- Draw the shape of things that we need to lift to lift a certain item.
- In the image, this is done for yellow item.
- This shape will always be an upside down triangle!



Subtask 4: All have rectangular shape

Let's (very slightly) reformulate the problem!

Every item has weight 1. What is the total weight we need to lift before we can lift a certain item?

For each item, arbitrarily choose one square where all of its weight is concentrated (weight = 1), and let all other squares have weight = 0.

Now, we can solve the problem if we can count the number of 1:s inside of the upside down triangle. Number of 1:s = sum of all values.

Subtask 4: All have rectangular shape

- Split triangle into two parts.
- Answer = Sum in green part minus sum in red part.

So how do we find these sums fast enough?

- In each row, (pre-)calculate prefix sums. This lets you ask “how many ones to the left of this position in this row?”
- Boundary of green region is inherited from rightmost parent
- Boundary of red region is inherited from leftmost parent.

Subtask 4: All have rectangular shape

Prefix sum:

Given these two rows
1 0 0 1 0 1
0 1 0 0 0 1
⟶ row-wise prefix sum
1 1 1 2 2 3
0 1 1 1 1 2

Subtask 4: All have rectangular shape

Let R[x] be area of green region for item x, and L[x] be area of red region. So ans[x] = R[x] - L[x] - 1 (subtract 1 to not count the item itself).

To calculate R[x] and L[x]:
Let pref = sum of all prefixsums along the right wall of x.
R[x] = R[rightmost parent] + pref
L[x] = L[leftmost parent] + pref - 1

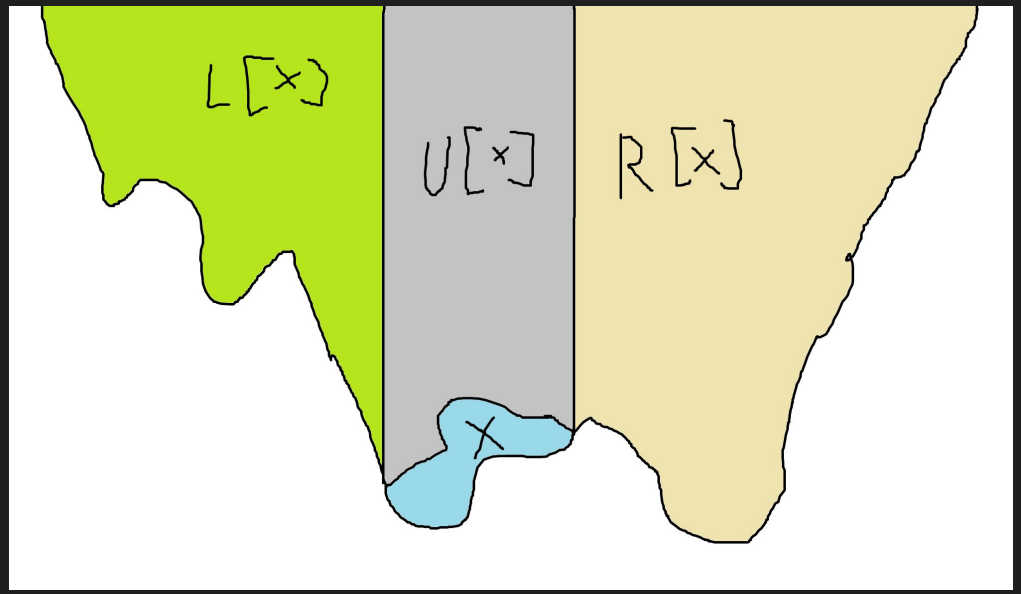
Full solution!

Same kind of thinking as rectangular subtask:
Split answer into different regions, one is inherited from rightmost parent, one is inherited from leftmost, answer is a combination of them.

Issue: region is no longer an upside down triangle: Imagine a comb-shaped item

Full solution!

- Divide “The stuff above x” into three regions as the image suggests
- Calculate L[x] using L[left parent of x]
- Calculate R[x] using R[right parent of x].
- Calculate U[x] with column-wise prefix sums

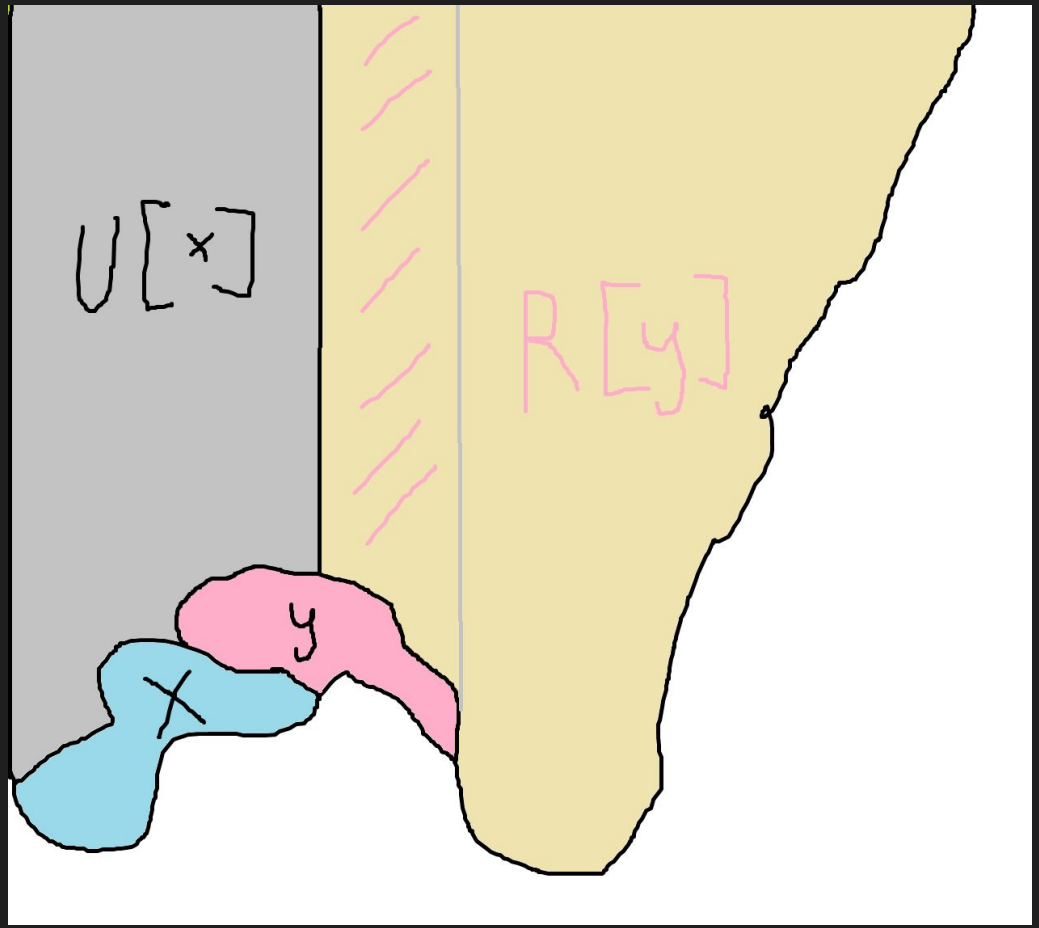


Full solution!

y = rightmost parent of x
R[x] = R[y] + shaded region

When calculating U[y], we can also build a row-wise prefixsum of the column-wise prefix sums which allows us to quickly query for the sum in the shaded region.

L[x] is calculated the same way.



Reseplaneraren

Av: Joshua Andersson

Subtask 1

Solve every query in O(NK): for each path, check if a and b are part of it

Subtask 2

Multiple possible solutions.

For each node, calculate which paths it is part of. To answer a query, check if the endpoints of the query share a path. O(K(N+Q))

Subtask 3

Optimize the solution to subtask 2 using bitsets. O(K(N+Q)/64)
Easy to answer queries, tricky to do the precomputation.
Do a DFS, maintain the currently active paths in a bitset. Turn on a certain path at either of its endpoints, disable it at the LCA of the endpoints.

Subtask 4

Let's solve this subtask in a way that generalizes to the full solution. In general, there are two cases: one endpoint is ancestor of the other, or not. We solve the is ancestor case.

For each node, calculate how far up we can go using a single path. Suppose WLOG that our query is from a to b and that b is deeper. Then, we can answer the query by checking if b can go up to a node closer the root than a.

Subtask 4

Calculating maxup[u] = how high in the tree can we get using a single path. We do a DFS. We say that a path is active if it contains the current node. For each active path, maintain how far up it can go. Maintain these in a sorted set. When we enter a node that is endpoint of path, add to set. When do we remove? At lowest common ancestor (LCA)

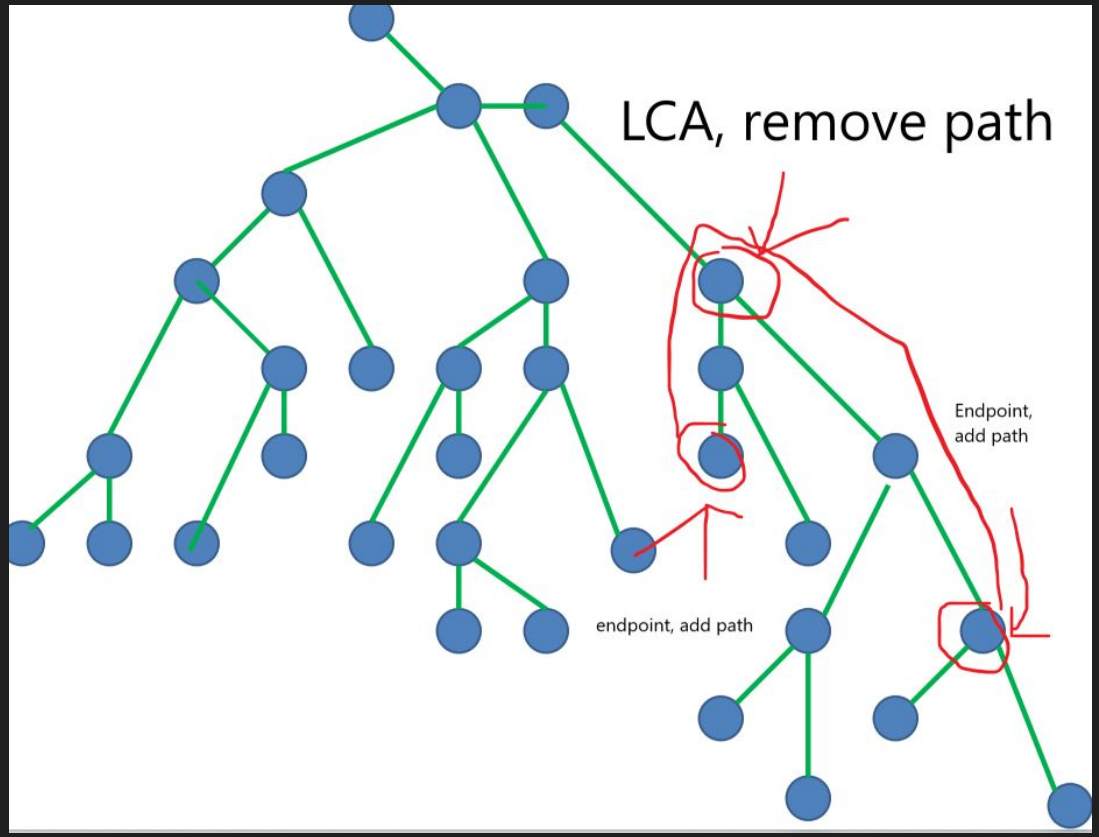
Om LCA och smaller to larger merging

https://docs.google.com/presentation/d/1vCWbYg-cQ2BSwqYwa8HFlupbgcGhDdnptGkos4fncjI/edit?usp=sharing

https://cp-algorithms.com/graph/lca_binary_lifting.html

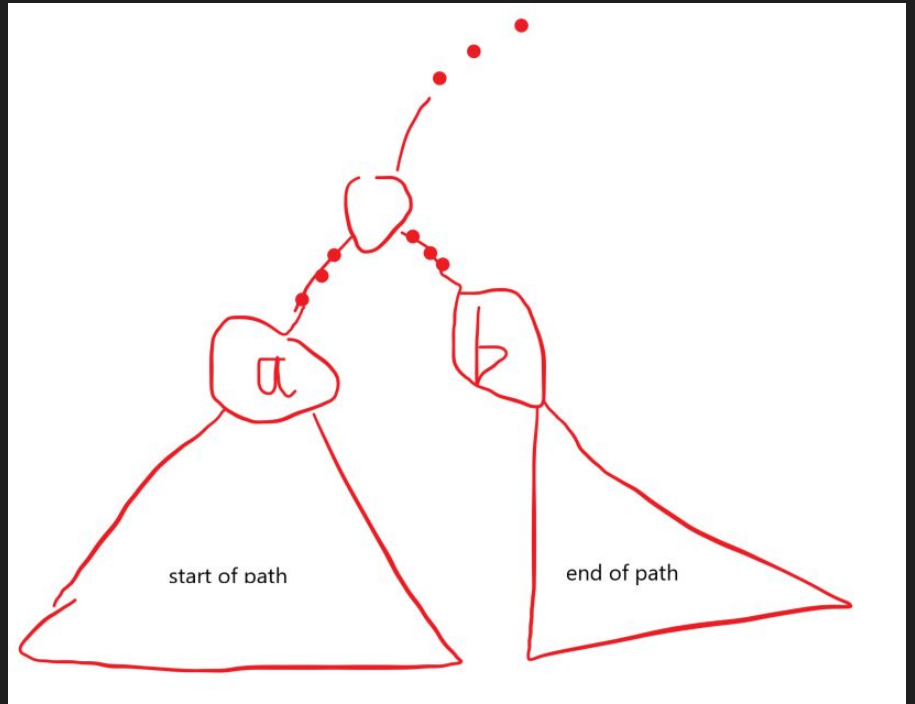
https://usaco.guide/plat/merging?lang=cpp

Subtask 4



Full solution

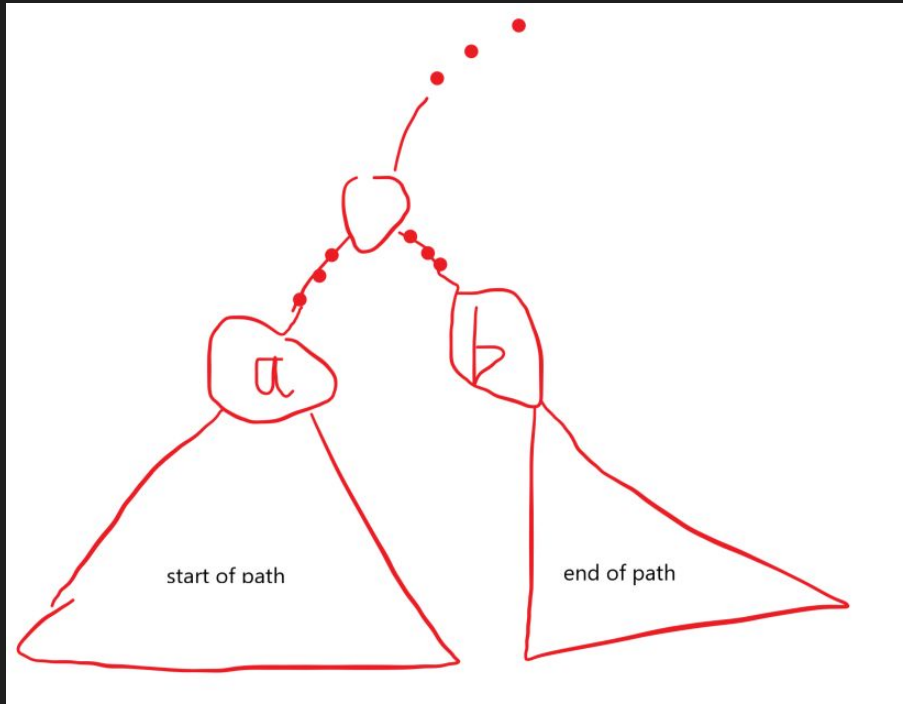
We know how to solve the case where one is ancestor of another. Now we need to find a path that starts in the subtree of a ends in the subtree of b.



Full solution

Denote the start of the path we're searching for by S and its end by E. Let ANC(a,b) = is a ancestor of b?

So we are searching for a path where ANC(a,S) and ANC(b,E) holds.



Full solution

How do we implement ANC(a,b)? Using euler tour

https://usaco.guide/gold/tree-euler?lang=cpp

Normal implementation is

```
bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}
```

However, following also works

```
bool isancestor(int a, int b)
{
    return tin[a] <= tin[b] && tin[b] <= tout[a];
}
```

Full solution

However, following also works

```
bool isancestor(int a, int b)
{
    return tin[a] <= tin[b] && tin[b] <= tout[a];
}
```

Notice how for b, only tin is used, not tout! For every path, create point (tin[S], tin[E]). To answer the query, we want to find a point where tin[a] <= tin[S] <= tout[a], tin[b] <= tin[E] <= tout[b]
If we create a 2D where x-axis is tin[a] and y is tin[b], this asks whether there is any point in the rectangle
[tin[a], tout[a]] x [tin[b], tout[b]]

Full solution

How to solve 2D rectangle queries? Easier to solve “count number of points in the rectangle”, and check if this is > 0.
Recommended way:
https://hackmd.io/@Matistjati/B1LdMKvdyg

Can either use 2D segment tree
https://usaco.guide/plat/2DRQ?lang=cpp

Resultat!